

A Report on the JavaScript Programming Language

Alex Chang, Daniel Coblenz

Hood College

CS471: Programming Languages

Dr. Aijuan Dong

December 6, 2025

Table of Contents	
Section 1: Introduction.....	3
1.1 Project goal	
1.2 Report overview	
Section 2: History.....	3
2.1 Introduction	
2.2 Creation and ECMAScript	
2.3 Lifetime	
Section 3: Basic features.....	4
3.1 Introduction	
3.2 Names, scopes, bindings	
3.3 Data types	
3.4 Control flow	
3.5 Subprograms	
3.6 Programming paradigms	
3.7 Miscellaneous	
Section 4: Distinctive language features.....	8
4.1 Objectives	
4.2 Meeting specifications	
Section 5: Tutorial.....	8
5.1 Introduction	
5.2 “Hello, world”	
5.3 Major features	
5.31 Hoisting	
5.32 Arrow functions	
5.33 Array destructuring	
5.34 ES6 modules: import and export	
5.4 Additional learning resources	
Section 6: Evaluation.....	11
6.1 Introduction	
6.2 Orthogonality	
6.3 Control structures	
6.4 Data types	
6.5 Syntax design	
6.6 Support for abstraction	
6.7 Expressivity	
6.8 Type checking	
6.9 Conclusion	
Section 7: References.....	14

Section 1: Introduction

1.1 Project goal

This report will provide a simple yet comprehensive overview of the JavaScript programming language. We chose JavaScript due to its versatility; as we will examine, JavaScript is a flexible and ubiquitous language. It is primarily used in both front-end and back-end web development but is seen outside of web pages (Mozilla). In addition, both of us have experience of working with it and are drawing from that experience in this report.

1.2 Report overview

We divide this report into many sections detailing various aspects of JavaScript, such as its history, the way it implements basic features, its design objectives, and how it meets them, a simple tutorial to the language and some unique features, and an evaluation based on eight major criteria.

Section 2: History

2.1 Introduction

This section outlines the history of JavaScript and the differences between it and ECMAScript. The two are often conflated, and the differences are not readily apparent. In addition, we look at how the language has developed and changed, and we examine its popularity and how it will grow in the future.

2.2 Creation and ECMAScript

JavaScript was created by Brendan Eich in 1995, originally under the name “Mocha”. It was developed for a now-defunct web browser named Netscape Navigator 2.0. Its predecessor, Netscape Navigator 1.0, was immensely popular as it had a graphical user-interface in contrast to the text-based web browsers in the early 1990s. Netscape tasked Eich with making a simple object-oriented scripting language like Java. This language would eventually be known as JavaScript and was designed to complement Java (Deno, 2025). In 1996, around the same time Netscape Navigator 2.0 was released with JavaScript 1.0, Microsoft developed JScript, a similar language to JavaScript but was used in Internet Explorer and integrated with other Microsoft products.

The schism between JavaScript and JScript presented a problem: a split developer community and a lack of cross-compatibility between web browsers. In 1997, Netscape submitted JavaScript to ECMA International who created a specification called ECMA-262 to standardize web development. These standards outline ECMAScript, which is a general-purpose programming language specification implemented in every major web browser of which JavaScript and JScript are dialects (ECMA International, 2025).

ECMAScript is a specification while JavaScript is the language; ECMAScript was made to standardize JavaScript and JScript and has continued to receive updates. The ECMAScript standard was officially released in 1997. The first update to ECMAScript would be ECMAScript

3, released in December 1999, and it implemented many critical features: regular expressions, string methods, and exception handling (Deno, 2025). This marked a turning point for JavaScript whose popularity exploded after this update.

Updates to ECMAScript continued after the turn of the millennium. The fourth edition would be scrapped, and some of its work being carried over to the fifth edition, published in 2009. This edition added support for JSON files, a strict mode with additional error checking and program security, extra array manipulation functions, and more (ECMA International, 2025). Its greatest edition, however, would be its sixth edition. ECMAScript 6, often shortened to ES6, intended to better support large-scale applications and more easily allow for library creation. Critical features of ES6 include class declarations, modules, iterators, and block-scoping, as well as plans for asynchronous programming, which would be added in ECMAScript 8 (ECMA International, 2025). ES6 was released in 2015, and editions of ECMAScript have since followed a yearly release schedule; the most recent version of ECMAScript is ECMAScript 2026: its seventeenth edition.

2.3 Lifetime

Developments to JavaScript go beyond developments to ECMAScript. AJAX, short for asynchronous JavaScript and XML, was created in 2005 by Jesse James Garrett. This allowed for web applications to both send and receive data without requiring a page reload; this was fundamental to many new applications and frameworks (Deno, 2025). In 2006, John Resig began working on jQuery, a library designed to simplify several aspects of web development with thorough documentation; it remains immensely popular today (Deno, 2025). Further developments to the language's extensive list of frameworks and libraries include Node.js in 2009: a runtime environment that allows JavaScript to be run outside of web browsers. This enabled developers to create servers and paved the way for full stack applications. 2009 also saw the creation of Express.js: a framework that allows for simple creation of RESTful APIs (Deno, 2025).

The 2010s would bring many critical developments to JavaScript. Frameworks such as AngularJS, React, and Vue.js were released in this decade, expanding the available tools for web developers. Node Package Manager, shortened to npm, was released and is now the default package manager for JavaScript. In 2012, a statically typed superset of JavaScript called TypeScript was made publicly available from Microsoft. The addition of static typing while still being compatible with JavaScript's dynamic typing makes it immensely popular today for writing large-scale applications. Additions to JavaScript's breadth of libraries and frameworks extend beyond what is mentioned here and continue to this day.

To say JavaScript is popular would be a substantial understatement. It has built an extensive network of developers and powers a staggering number of applications while still being easy to use. The TIOBE index ranks JavaScript at 6th, behind Python, C, C++, Java, and C#, based on the number of hits from multiple top search engines (TIOBE, 2025). The history of JavaScript certainly corroborates such a placement.

Section 3: Basic features

3.1 Introduction

In this section, we outline basic features common to many programming languages and observe their implementation in JavaScript.

3.2 Names, scopes, bindings

ES6 provides two ways of declaring variables: *let* and *const*, which act as replacements to the ES5 way of declaring variables: *var*. *Var* still sees use in declaring variables but due to being solely function-scoped, it is rarely used now.

The keyword *let* works similarly to *var*, but the variable it declares is block-scoped, so it only exists within the current block whereas the *var* keyword creates function-scoped variables. Additionally, *const* works like *let* but the declared variable must be immediately initialized with a value that cannot be changed after it is declared. This is useful for defining equations or values that the programmer knows will remain unaltered during runtime, but this works differently with primitives than it does with objects. We explore primitive data types and objects in section 3.3, but for this section's purposes we will only look at these categories, not individual data types. Primitives are immutable by default regardless of *const* when declared, but *const* only prevents reassignment, while objects can change their value, but they will always reference the same location in memory. (Mozilla, n.d.)

Both *let* and *const* create variables that are block-scoped, meaning they exist only inside the nearest set of curly braces that encloses them. In contrast, *var* is function-scoped, so it is available throughout the entire function in which it is declared. Additionally, variables declared with *let* and *const* enter a temporal dead zone (TDZ): they cannot be accessed within their scope until execution reaches their declaration (Rauschmayer, 2018).

JavaScript uses bindings to connect variable names to state locations in memory. A declarative environment record binds the set of identifiers defined by the declarations contained within its scope (ECMA International, 2025) when you declare a variable with *var*, *let* or *const* the JavaScript engine creates a binding in an environment record, which is the structure that tracks all variables and their values within a given scope.

3.3 Data types

JavaScript has eight data types primarily split into two categories: primitives and objects. The seven primitive types are *number*, *string*, *Boolean*, *undefined*, *null*, *symbol* and *BigInt*. The eighth type is *object*, which includes arrays, functions, and other complex data structures (Mozilla, n.d.). We begin this section by talking about primitives as a whole as well as JavaScript's dynamic typing, then discussing individual data types.

Primitives store values directly. When a variable has a primitive value, JavaScript copies the value. For example, “let a = 10” defines a variable *a* and sets the value to 10, and then “let b = a” will create a new value *b* with the same value; these are two separate values in memory. Changing *b* will not affect *a* because they are independent copies. This is different than objects,

which store references to locations in memory. When an object is assigned to another variable, both variables point to the same object. For example, “let obj1 = {count: 10}” will create *obj1* with a *count* value of 10. Next, “let obj2 = obj1” will create *obj2* which references the same object in memory. Changing *obj2.count* will also change *obj1.count* because they are the same object (Mozilla, n.d.).

JavaScript uses dynamic typing, which means variables can hold any type and change types during execution. For example, a variable can be assigned to a number then reassigned to a string, then to an object without any declaration of changes. This creates flexibility but also introduces risks. The language also performs an automatic type coercion in certain operations. As an example, the statement “ “5” + 3” will have the numeric value of 3 converted into a string, which produces the result of “53”. When you write “ “5” - 3”, it converts the string to a number then produces a numeric value of 2. These conversions follow rules that may not be intuitive for inexperienced users of the language. (GeeksForGeeks, 2025).

The *Number* type uses a 64-bit floating point format for all numeric values. This includes both integers and decimals. The *BigInt* type was added later to handle larger integers beyond the standard range. *Strings* can use single quotes, double quotes, or backticks, enabling template literals for interpolation. *Boolean* values can only be true or false. *Undefined* means a variable exists, but there is no assigned value to it, while *null* represents an intentional absence of a value. Lastly, *Symbol* creates a unique identifier that will not collide with other property names. This can be especially helpful for making packages or libraries for other developers to use. (Mozilla, n.d.)

3.4 Control flow

JavaScript implements three common control structures: conditional statements, loops, and exception handling. Conditional statements change control flow based on the value of a conditional statement. The familiar *if-else* statement is an example of a conditional statement. These statements run based on a condition, and the corresponding block in the statement runs when its condition is met.

JavaScript also implements *for*, *while*, and *do-while* loops, all of which allow for the repetition of a block of code until a specific condition is met. *For* loops execute a block of code while updating a value until the value reaches its end condition. *While* and *do-while* loops are similar and run a block of code until a specified condition is met without directly updating a value, but *while* loops check this condition before executing, *do-while* loops run before checking their finishing condition.

Exception handling in JavaScript works similarly to many other popular programming languages. The *throw* statement allows the programmer to generate, or throw, exceptions or errors that can interrupt program execution. Anything can be thrown in JavaScript, but there are specific exception objects for this purpose (Mozilla, n.d.). The *try* and *catch* statements allow the programmer to specify code to run in the *try* block, and responses to be thrown given a condition in the *catch* block.

3.5 Subprograms

Subprograms are implemented as functions, which can return a value but do not necessarily have to. Functions can take parameters, though something unique to JavaScript is that it does not require the exact number of parameters specified to run the function; it will ignore excess parameters and set unspecified parameters to *undefined* (Mozilla, n.d.). Recursion is also implemented in JavaScript as functions can call themselves. It is worth noting, however, that even though ECMAScript specifies that all recursive calls are tail-recursive, this is not universal (ECMA International, 2025). Functions can be implemented inside other functions and do not always require names. We will explore inline arrow functions in section 5.32 as an example of functions that do not require names.

3.6 Programming paradigms

JavaScript is a multi-paradigm language that supports multiple programming styles. It supports object-oriented programming where programs are a collection of objects that interact with one another. Unlike class-based languages such as Java or C++, JavaScript is fundamentally prototype-based; every object inherits properties through a prototype chain, and this mechanism forms the core of its inheritance model (Mozilla, n.d.).

JavaScript also supports functional programming, where functions are treated as first-class values and programs are built by composing small, reusable operations rather than relying on shared state. In addition, JavaScript supports imperative and procedural programming styles, giving developers the flexibility to choose the paradigm that best fits their problem domain. This versatility allows JavaScript to adapt to many different development approaches (Yusuf, 2020).

3.7 Miscellaneous

JavaScript is single threaded, meaning it executes code on a single call stack and does not directly support parallel execution of multiple threads. However, JavaScript implements asynchronous programming through *async* and *await* keywords, promises and callback functions, allowing concurrent operations without actual parallelism. This concurrency model allows JavaScript to handle many I/O operations, network requests, and timers without disrupting the main execution thread, making it a perfect choice for event driven programming in web browsers and Node.js environments (Mozilla, n.d.).

Although the main thread is single-threaded, JavaScript environments can achieve true parallelism using web workers in browsers and worker threads in Node.js. These execute JavaScript on separate threads and communicate with the main thread through message passing. The event loop manages queued tasks and microtasks, enabling JavaScript to handle asynchronous operations efficiently while long-running work is executed in the background (Mozilla, n.d.).

Section 4: Distinctive language features

4.1 Objectives

JavaScript was designed to be ECMAScript-compliant and implements additional features that ECMA-262 does not outline. JavaScript is primarily meant to provide client-side interactivity for web browsers. JavaScript began as a scripting language but is now integrated into every popular browser after 2018 (W3 Schools, n.d.). As such, it provides many features for this purpose, but libraries and frameworks exist to run JavaScript code client-side such as Node.js.

4.2 Meeting specifications

JavaScript has seen widespread adoption due to its adherence to ECMAScript, which ensures consistency and interoperability across different JavaScript implementations and environments. This compliance is a major reason for its adoption: browsers, Node.js and other runtimes all rely on the same core standard, allowing developers to write JavaScript that runs predictably regardless of platform (Mozilla, n.d.).

As ECMAScript evolved from early editions to ES5 and especially ES6, JavaScript implementations have steadily incorporated new features such as classes and modules. Modern browsers update frequently to stay aligned with the yearly updates to the specification, and Node.js versions track the same yearly ECMAScript releases. This cycle allows JavaScript to maintain interoperability while still adding modern capabilities.

In practice, meeting specifications means that JavaScript engines such as V8 and JavaScriptCore all follow the same shared blueprint for syntax, semantics, and runtime behavior. Features such as lexical scoping, strict mode, number handling, and function behavior are all governed by the ECMAScript standard because of these developers can rely on ECMAScript as the point of control defining what valid JavaScript is and how programs should execute.

Section 5: Tutorial

5.1 Introduction

This tutorial demonstrates JavaScript's core functionality and distinctive features through a few examples. We begin by creating a “hello world!” program. Following this, we explore three key features that separate it from other programming languages: hoisting, arrow functions, and array destructuring. These features illustrate JavaScript’s flexibility and modern programming paradigms. Each example includes explanations to give readers a thorough understanding of both the syntax and the underlying mechanisms that make JavaScript a powerful tool for web development. The examples presented can be executed in any modern browser console or in a Node.js environment, making them accessible for hands-on experimentation.

5.2 “Hello, world”

A “hello, world!” program in JavaScript can be written in one line: `console.log(“Hello, world!”)`. `Console.log` is a function in JavaScript which outputs information to the console. Its primary use case is debugging and inspecting the state of a program during development.

5.3 Major features

5.31 Hoisting

Hoisting refers to the JavaScript interpreter moving declarations to the top of their scope before code execution begins. While not formally recognized as a feature in the ECMAScript specification, hoisting has been an inherent behavior of JavaScript since its creation in 1995. The term “hoisting” emerged later as colloquial terminology to describe this mechanism. This behavior can be counterintuitive for programmers coming from languages like C++ or Java, where declarations must precede their use. Although JavaScript code executes sequentially, or line by line, hoisting makes it appear non-sequential because the interpreter processes all declarations during an initial phase before executing any statements. This mechanism allows references to variables and functions before they appear in the source code. Below is a simple example of hoisting.

```
    \ \ \
sayHello();

function sayHello() {
    console.log(“Hello!”)
}
    \ \ \
```

In this example, we demonstrate hoisting by calling the *sayHello()* function before it is defined in the code. Despite the function declaration appearing below the function call, the program executes successfully. This occurs because JavaScript hoists the entire function declaration to the top of the scope during the compilation phase, making it available throughout the entire scope. The interpreter processes this code as if the function declaration were written at the top, allowing the function call on the first line to execute without error. This behavior is unique to function declarations and demonstrates how hoisting can make code execution appear non-sequential.

5.32 Arrow functions

Arrow functions are semantically different to typical function declarations, where they do not require a name and can take any number of parameters. They can be defined using a more concise syntax, making them useful for inline functions and callbacks. In ES6,

arrow functions provide a shorter alternative to traditional function expression while also handling the *this* keyword differently.

The syntax of an arrow function uses the “=>” operator. Below are three examples with the traditional declaration, arrow function equivalent, and the concise arrow function equivalent.

```
```\n// 1) traditional function expression\nconst add = function(a,b) {\n  return a + b;\n}\n\n// 2)\nconst add = (a,b) => {\n  return a + b;\n}\n\n// 3) concise arrow function (implicit return)\nconst add = (a,b) => a + b;\n```\n
```

Arrow functions differ from traditional function declarations in important ways. They do not have their own *this* binding. Instead, they inherit *this* from the surrounding lexical context. This makes them ideal for use in callbacks where maintaining the correct *this* context is crucial. Additionally, arrow functions cannot be used as constructors and cannot access the arguments of objects available in traditional functions.

### 5.33 Array destructuring

Array destructuring refers to mapping values in an array or object to distinct variables. Introduced in ECMAScript 6, this feature provides a concise syntax for extracting multiple values from data structures in a one-line statement

The syntax uses square brackets on the left side of an argument to specify which values to extract. If we only want to extract specific values, we can skip elements using commas representing those skipped positions. An example of array destructuring is shown below in contrast to the traditional approach to mapping values from an array to unique variables.

```
```\n\n// Traditional approach\nconst colors = ["red", "green", "blue"];\nconst first = colors[0];\nconst second = colors[1];\nconst third = colors[2];\n```\n
```

```
// Array destructuring
const [first, second, third] = colors;
console.log(first); // Output: "red"
console.log(second); // Output: "green"
console.log(third); // Output: "blue"
““
```

5.34 ES6 Modules: import and export

ES6 introduced native modules system to help developers organize code into smaller, reusable files. A module is a JavaScript file that explicitly exports values such as functions variables or classes which can be imported into other files. This system improves maintainability in large applications and prevents variables from crowding the global scope.

To export a value, you use the *export* keyword, and to use that value in another file, you import it with the *import* statement. Modules can also perform default exports, allowing a file to export a single main value browser and Node.js both support ES6 modules (ECMA International, 2025).

5.4 Additional learning resources

For more information about JavaScript’s syntax and features, consult the Mozilla Web Documentation, linked in section 7; this covers the bulk of information about JavaScript.

Section 6: Evaluation

6.1 Introduction

This section evaluates JavaScript using several of the criteria commonly applied to programming languages. Instead of focusing only on its syntax or history, we look at how well JavaScript supports core language principles such as simplicity, orthogonality, abstraction, expressivity, and type checking. Evaluating the language in this way helps highlight both the core strengths such as flexible syntax and support for multiple paradigms and its limitations such as weak typing. Together, these criteria give a complete picture of how JavaScript functions as a modern, general-purpose scripting language.

6.2 Orthogonality

Some, but not all, features in JavaScript are orthogonal. Two examples of orthogonal features, functions can be used anywhere a value is used; they are consistent in this way. Array methods will not work on array-like objects without converting them to arrays (Mozilla, n.d.). By contrast, *var*, *let*, and *const* illustrate the non-orthogonal nature of variable declaration. Both *let* and *const* are block-scoped while *var* is function-scoped, and *var* declarations get hoisted while *let* and *const* do not; this inconsistency is a small difference but can be challenging for newcomers to grasp.

6.3 Control structures

JavaScript provides a similar set of control structures that match the style of many C-based languages, making it easy for developers to follow common procedural patterns. It supports conditional statements as *if-else*, and *switch*, along with iterative structures such as *for*, *while* and *do-while* loops. These constructs behave predictably and form the backbone of most JavaScript programs.

JavaScript also includes more modern control flow tools, such as *for...of* for iterables *for...in* for object properties, giving developers flexibility depending on the data type they are working with. Support for *try*, *catch* and *finally* allows programs to efficiently handle runtime errors, which is a core component in web environments where unexpected inputs and asynchronous events are common (Mozilla, n.d.).

Overall, JavaScript's control structures are straightforward and easy to follow syntactically, offering both traditional looping mechanisms and newer, more expressive options suited for everyday development of software and programs.

6.4 Data types

JavaScript supports multiple built-in data types; these include primitive types such as *Number*, *String*, *Boolean*, *BigInt*, *Symbol*, *null* and *undefined* along with the *object* type, which covers arrays, functions, and user defined structures. JavaScript is dynamically typed, variables can hold values of any type without explicit declaration, and their types can change at runtime.

While this flexibility makes the language easy to use, it also means developers must be careful about implicit type conversions. JavaScript frequently performs automatic coercions in comparisons and arithmetic which can lead to behavior that feels inconsistent to newcomers. Despite this, the dynamic type of system is central to how JavaScript is used in web development, enabling rapid development and prototyping with data from user input API's and the document object model.

6.5 Syntax design

JavaScript's syntax follows a C-style structure making it automatically familiar to those with experience in languages like C++, Java, or C. It is dynamically typed and case-sensitive, allowing developers to write code quickly without declaring the exact types. Statements typically end with a semicolon, which JavaScript's automatic semicolon insertion can infer to them in many situations. The syntax itself is simple to read, with clear patterns for defining functions, objects, loops, and conditions.

This design contributes to the ease of adoption of the language while still supporting more advanced features through additions made in ES6 such as arrow functions, template literals, classes, and modules. The combination of familiar syntax and modern enhancements makes the language approachable for beginners while providing enough expressive power for large-scale applications.

6.6 Support for abstraction

JavaScript implements functions and classes. There are several mechanisms for abstraction, allowing developers to manage complexity as applications grow. Functions act as the primary abstraction tool and can encapsulate logic, accept parameters, and return values. The introduction of ES6 allows for the support of class-based syntax, which provides the option to define objects with shared behavior using constructors, methods, and inheritance. Although the language is fundamentally prototype-based, the class's syntax offers a cleaner, more structured approach to organizing code. Functions, closures, modules, and classes give JavaScript support for abstraction across both small- and large-scale codebases.

6.7 Expressivity

JavaScript is a highly expressive language, offering concise syntax and multiple ways to accomplish the same task. Features like arrow functions, template literals, destructuring, and first-class functions allow people to write clear and compact code. Its dynamic typing and flexible object models also make it easy to work with complex data structures or create new ones instantly. This expressiveness paints JavaScript as beginner-friendly while still being powerful enough for professional-level development.

6.8 Type checking

JavaScript uses dynamic type checking; meaning types are validated at runtime rather than during compilation. This approach gives professionals flexibility and speeds up early development but can introduce subtle bugs when values are coerced between types implicitly. For example, comparisons using `==` may trigger automatic conversions that behave differently than expected, while arithmetic operators can shift types depending on the operands involved.

To address these issues, many developers adopt TypeScript, a superset of JavaScript that adds static type checking and catches type errors before execution. While JavaScript itself does not enforce compile-time type constraints, the ecosystem provides tools and compensates for this gap when stronger guarantees are needed.

6.9 Conclusion

JavaScript has evolved from a simple browser scripting language to a general-purpose tool used across every area for modern software development. Its long history, guided by ECMA, has allowed it to maintain consistency across platforms while steadily introducing new features that support today's programming demands. The core characteristics explored in this report its flexible syntax, dynamic typing, multi-paradigm design, support for abstraction, and expressive features. All of these illustrate why JavaScript remains central to both frontend and backend development.

Although its dynamic typing system and occasional inconsistencies present challenges, the language ecosystem, libraries, and modern additions like classes, modules, and asynchronous programming have improved its reliability and capability. Overall, JavaScript continues to

succeed because it balances accessibility for beginners with the depth needed for large-scale applications, making it one of the most influential languages in software history.

Section 7: References

Deno. (2025, May 22). *A brief history of JavaScript*. Deno.
<https://deno.com/blog/history-of-javascript>

ECMA International. (2025, December 4). *ECMAScript® 2026 Language Specification* (S. Gao, M. Ficarra, & K. Gibbons, Eds.). TC39; ECMA International.
<https://tc39.es/ecma262/#sec-normative-references>

GeeksforGeeks. (2025a, July 27). *History of JavaScript*. GeeksforGeeks.
<https://www.geeksforgeeks.org/javascript/history-of-javascript/>

GeeksforGeeks. (2025b, August 31). *Introduction to JavaScript*. GeeksforGeeks.
<https://www.geeksforgeeks.org/javascript/introduction-to-javascript/#>

Google. (n.d.). *Documentation*. V8.Dev. Retrieved December 6, 2025, from
<https://v8.dev/docs>

jQuery Foundation. (n.d.). *History | jQuery Foundation*. JQuery Foundation. Retrieved December 6, 2025, from <https://jquery.org/history/>

Mozilla. (n.d.). *JavaScript*. MDN Web Docs. Retrieved December 6, 2025, from
<https://developer.mozilla.org/en-US/docs/Web/javascript>

OpenJS Foundation. (n.d.). *The V8 JavaScript Engine*. Nodejs.org. Retrieved December 6, 2025, from <https://nodejs.org/en/learn/getting-started/the-v8-javascript-engine>

Rohith, N. (2025, November 19). *Inside Node.js: A Deep Dive into V8, libuv, the Event Loop & Thread Pool*. DEV Community. https://dev.to/rohith_nag/inside-nodejs-a-deep-dive-into-v8-libuv-the-event-loop-thread-pool-5fcn

Shaikh, A. (2024, July 16). *Deep Dive into JavaScript Closures: How and When to Use Them*. DEV Community. <https://dev.to/itsshaikhaj/deep-dive-into-javascript-closures-how-and-when-to-use-them-5c63>

W3 Schools. (n.d.). *JavaScript History*. Www.w3schools.com. Retrieved December 6, 2025, from https://www.w3schools.com/js/js_history.asp